CONTRIBUTOR GUIDE

Contributing to Power Glove Vision

Source style, testing, documentation, packaging, and pull-request expectations.

2026 EDITION / IAIN BENNETT / MIT LICENSE

Thank you for helping improve PowerGlove Vision. Keep changes focused, readable, and safe for a project that combines camera input, local networking, virtual Linux devices, and a privileged shutdown helper.

Before changing code

1. Start from the current `dev` branch and create a short-lived topic branch.
2. Read `SECURITY.md` before changing pairing, tokens, network listeners, downloads, file permissions, uinput, or shutdown behavior.
3. Check `CONFIGURATION_REFERENCE.md` before changing a file format, default, installed path, port, controller mapping, or service unit.
4. Never commit `data/`, tokens, passwords, pairing codes, the downloaded hand model, caches, or locally generated App Lab installation ZIP files.

Security vulnerabilities should follow the private reporting process in `SECURITY.md`, not an ordinary pull request or public issue.

Branch and release workflow

`dev` is the integration branch for ordinary development. Create feature, fix, and documentation branches from current `dev`, then open pull requests back to `dev`. Do not commit ordinary development directly to `main`.

`main` represents released code. To prepare a release:

1. Confirm the complete quality workflow passes on `dev`.
2. Finalize the version and move the release notes out of `Unreleased`.
3. Open and review a pull request from `dev` to `main`.
4. Merge only the reviewed release changes, tag the release, and verify its generated App Lab package.
5. Merge any release-only adjustments on `main` back into `dev` immediately.

For an urgent released-version fix, branch from `main`, review and merge the fix into `main`, publish the corrective release, and then merge `main` back into `dev`. Keep both branches protected against unreviewed or failing changes when the repository host supports branch protection.

Source style and documentation

Every executable, source file, service unit, and comment-capable application configuration must begin with the standard project header. Preserve a shebang as the first line when one is required. The header must identify:

- project and repository-relative filename;
- concise purpose;
- author and copyright;
- `SPDX-License-Identifier: MIT`;
- a short dated change log;
- `docs/CHANGELOG.md` and Git as the complete history.

Python modules need a useful module docstring. Public classes and functions, plus non-obvious security, protocol, lifecycle, and numerical helpers, need focused docstrings describing their behavior and safety conditions. Comments should explain decisions and constraints rather than repeat the next line of code.

Use four-space Python indentation, type annotations for interfaces, descriptive names, bounded network and file operations, and existing project patterns. Shell scripts use the shell declared by their shebang; Bash scripts should keep `set -euo pipefail`. Avoid adding a dependency when the standard library or an existing dependency handles the requirement clearly.

JSON does not accept comments. Keep JSON files valid and document their fields, consumer, installed location, defaults, and secret status in `CONFIGURATION_REFERENCE.md`.

Run the source audit before committing:

```sh
scripts/check-source-docs.py
```

Tests and checks

Core tests must remain independent of a physical camera, UNO Q, and RetroPie:

```sh
PYTHONPATH=src python3 -m unittest discover -s tests -v
python3 -m compileall -q python scripts src tests
```

Run the documentation and syntax checks:

```sh
scripts/check-documentation.py
for file in scripts/*.sh; do bash -n "$file"; done
for file in retropie/bin/* retropie/*.sh; do sh -n "$file"; done
```

Add or update tests when behavior, validation, security boundaries, packet formats, gesture mappings, configuration parsing, or recovery paths change. Do not add tests that only repeat static configuration without protecting a meaningful contract.

Documentation changes

Project Markdown belongs under `docs/`, except the repository-root `README.md`. Update all affected guides when a user-visible command, path, control, screen, configuration field, dependency, or troubleshooting procedure changes.

Add user-visible work to the appropriate category under `Unreleased` in `docs/CHANGELOG.md`. Git history remains the exact implementation record; do not paste a full Git log into individual source headers.

Rebuild and inspect all PDF editions after changing maintained Markdown:

```sh
python3 scripts/build-docs-pdf.py
scripts/check-documentation.py --require-pdfs
```

Treat Markdown as the documentation source of truth. Commit the updated Markdown and regenerated PDFs together, even when the edit appears small. Inspect the affected PDF pages for clipped text, broken tables, missing images, and unintended page breaks before committing.

The public `README.md`, guides under `docs/`, and documentation images also drive the Help Center hosted by the UNO Q. After a documentation change is merged or otherwise ready to deploy, synchronize and verify that copy with:

```sh
scripts/deploy-uno-q-wifi.sh arduino@UNO-Q-NAME.local
```

That deployment restarts the Power Glove Vision application and checks every Help route, every public PDF, and representative gesture artwork. Confirm the affected page and its **Open PDF** link in a browser after deployment. The cabinet-specific `docs/cheatsheet.md` and its quick-reference PDF are intentionally excluded from the public UNO Q Help deployment. If the UNO Q is unavailable, state clearly that device synchronization and live Help verification remain outstanding.

App Lab installation package changes

Build and verify the model-free App Lab installation ZIP with:

```sh
scripts/build-app-lab-package.sh
scripts/verify-app-lab-package.py
```

The App Lab installation ZIP must contain public source, configuration examples, documentation, the nine allowlisted public PDF guides, and the required custom UNO Q MediaPipe wheel. It must exclude private `data/`, the downloaded Google model, tests, the cabinet quick-reference PDF, caches, and Git history. Do not force-add the generated ZIP to Git. GitHub Actions publishes a short-lived verified ZIP artifact for each successful workflow run; tagged releases may attach a verified ZIP for long-term distribution.

Commits and pull requests

- Use a short imperative commit subject that describes the outcome.
- Explain what changed, why it was needed, and how it was verified.
- Keep unrelated cleanup separate from behavioral changes.
- Call out migration, deployment, compatibility, security, and rollback risks.
- Do not claim hardware verification unless the change was tested on the named device. Automated tests and simulated input should be described accurately.
- Wait for the GitHub Actions quality workflow to pass before merging.
- Target ordinary pull requests at `dev`; reserve pull requests into `main` for reviewed releases and urgent release fixes.